



StockPulse

A Multi-threaded Portfolio & Market Simulation System

Course	CSE2006 — Programming in Java
Assignment	VITyarthi — Build Your Own Project
Student	Aadarsh
Registration	25BAI11087
Institution	VIT Bhopal University
Date	September 2026

Table of Contents

1. Introduction
2. Problem Statement
3. Functional Requirements
4. Non-Functional Requirements
5. System Architecture
6. Design Diagrams
7. Design Decisions & Rationale
8. Implementation Details
9. Screenshots / Results
10. Testing Approach
11. Challenges Faced
12. Learnings & Key Takeaways
13. Future Enhancements
14. References

1. Introduction

StockPulse is a console-based Java application built for the CSE2006 (Programming in Java) "Build Your Own Project" assignment. It simulates a single investor managing a multi-asset portfolio — stocks, cryptocurrencies, and mutual funds — against a market whose prices move continuously in the background while trades are placed.

The project was deliberately scoped to exercise, in one coherent domain, every unit of the CSE2006 syllabus: object-oriented design (Unit 2), exception handling and multithreading with real synchronization (Unit 3), the Collections framework and I/O streams (Unit 4), and JDBC-based persistence (Unit 5). Rather than bolting these on separately, each concept is load-bearing: the market simulator genuinely needs a background thread, the trading engine genuinely needs synchronization to avoid a race between a price update and an order, and the JDBC layer genuinely needs to survive an application restart.

2. Problem Statement

Retail and student investors holding a small, mixed portfolio have no lightweight way to track holdings and profit/loss without spreadsheet arithmetic or a full brokerage account they don't need. Separately, most classroom demonstrations of Java concurrency use a toy counter, which does not show a student *why* synchronization matters — nothing visibly breaks if it's done wrong. StockPulse addresses both: it is a small, self-contained portfolio tracker, and its core trading loop is a scenario where an unsynchronized read of a live-changing price would produce an incorrect trade.

Scope

In scope	Out of scope
Single-portfolio, single-user simulation; three asset types (stock, crypto, mutual fund); a simulated live price feed; a text console interface; JDBC persistence.	Graphical user interface; a real market-data API integration; multiple concurrent user portfolios.

Target Users

Students learning investment concepts who want a safe, zero-risk simulation, and anyone wanting a minimal, self-hosted way to track a small multi-asset portfolio.

3. Functional Requirements

The brief requires at least three major functional modules with a clear input/output structure and a logical user workflow. StockPulse implements exactly this as three modules:

#	Module	Responsibility
1	Portfolio & Asset Management	Maintains the asset catalogue and the investor's cash + holdings; persists both via JDBC (<code>AssetDao</code> , <code>PortfolioDao</code>).
2	Trading Engine	Validates and executes buy/sell orders against live prices; rejects invalid orders with a specific, typed exception (<code>TradingEngine</code>).
3	Market Simulation & Analytics	Runs the background live-price thread and computes net worth, per-holding P&L, and best/worst performer (<code>MarketDataSimulator</code> , <code>AnalyticsService</code>).

Every module exposes a clear input (console menu choice + typed arguments) and output (console text, a persisted database row, or a CSV/log file), and the console menu enforces one logical workflow: view state → place an order → see the updated state or a rejection reason.

4. Non-Functional Requirements

Requirement	How it is addressed
Performance	Price reads/writes use a lock-free <code>AtomicReference</code> per asset instead of a shared lock, so the simulator ticking every few seconds never blocks a trade and vice versa.
Reliability	Every buy/sell either fully applies (cash debited, holding updated, transaction logged) or fully fails with an exception — there is no code path that leaves the portfolio in a half-updated state.
Security	All SQL is parameterized through <code>PreparedStatement</code> ; no string-concatenated SQL exists anywhere in the DAO layer, eliminating SQL injection by construction.
Usability	A single numbered menu with consistent prompts; every rejected action prints a specific, actionable reason instead of a stack trace.
Maintainability	Strict package-by-responsibility layout (<code>model</code> , <code>exception</code> , <code>dao</code> , <code>service</code> , <code>market</code> , <code>io</code> , <code>cli</code>) so each concern can be read, tested, and changed independently.
Logging / Auditability	Every executed trade is appended to both the <code>transactions</code> database table and a plain-text <code>data/trade_log.txt</code> audit file.

5. System Architecture

The application is layered so that the frequently-changing, in-memory market state stays off the database's hot path, while trades — infrequent but must never be lost — are written through immediately.

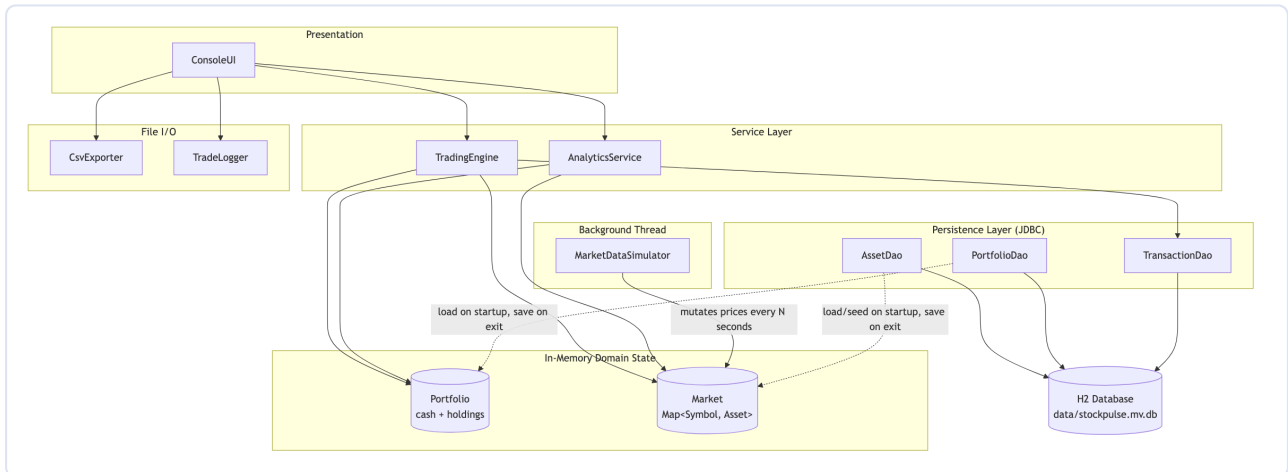


Figure 1 — Layered architecture. Solid arrows are method calls; the shaded persistence layer is touched only at startup, on each trade, and at shutdown — never on every simulator tick.

6. Design Diagrams

6.1 Use Case Diagram

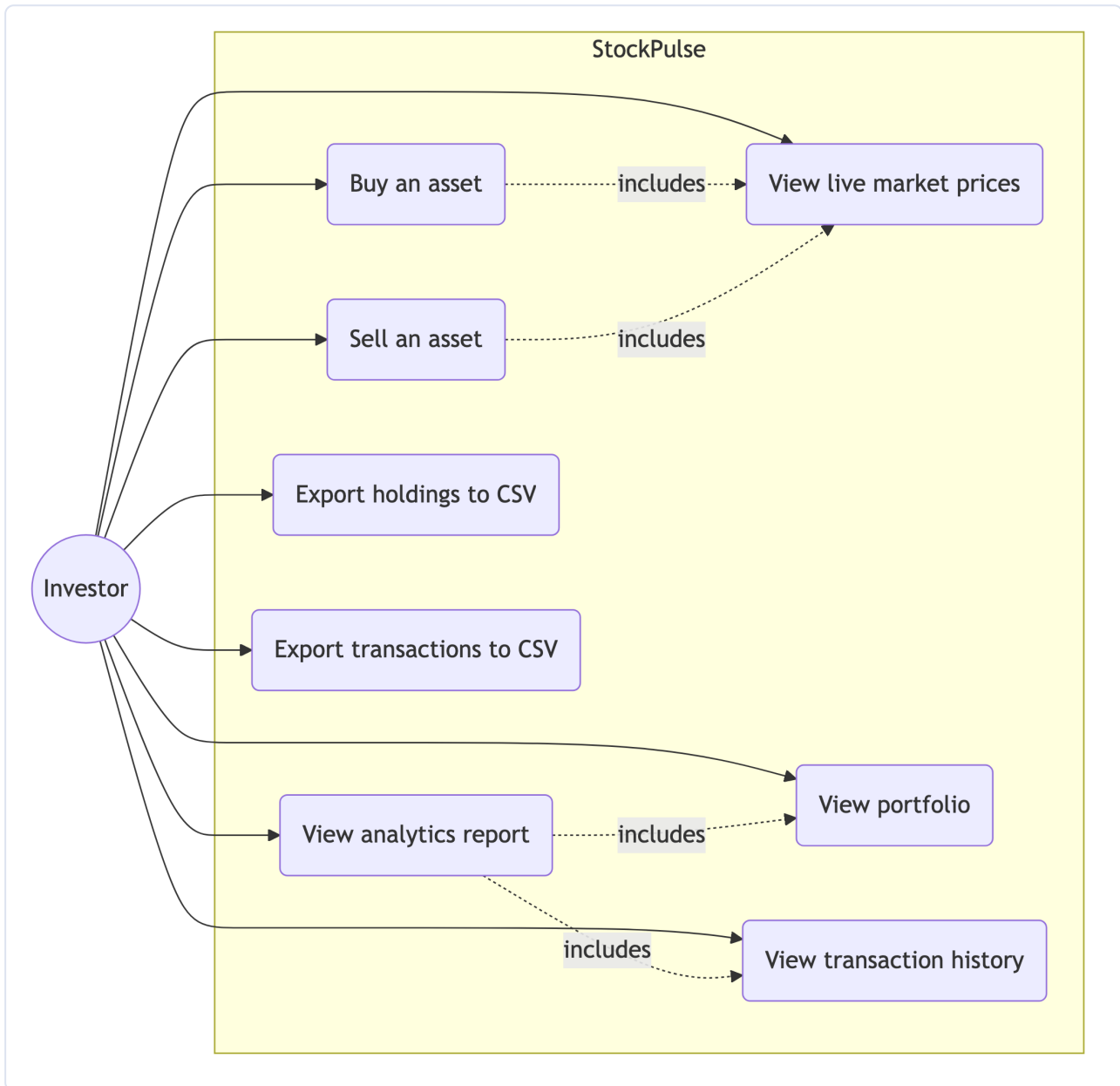


Figure 2 — All eight investor-facing actions the console exposes.

6.2 Process Flow / Workflow Diagram

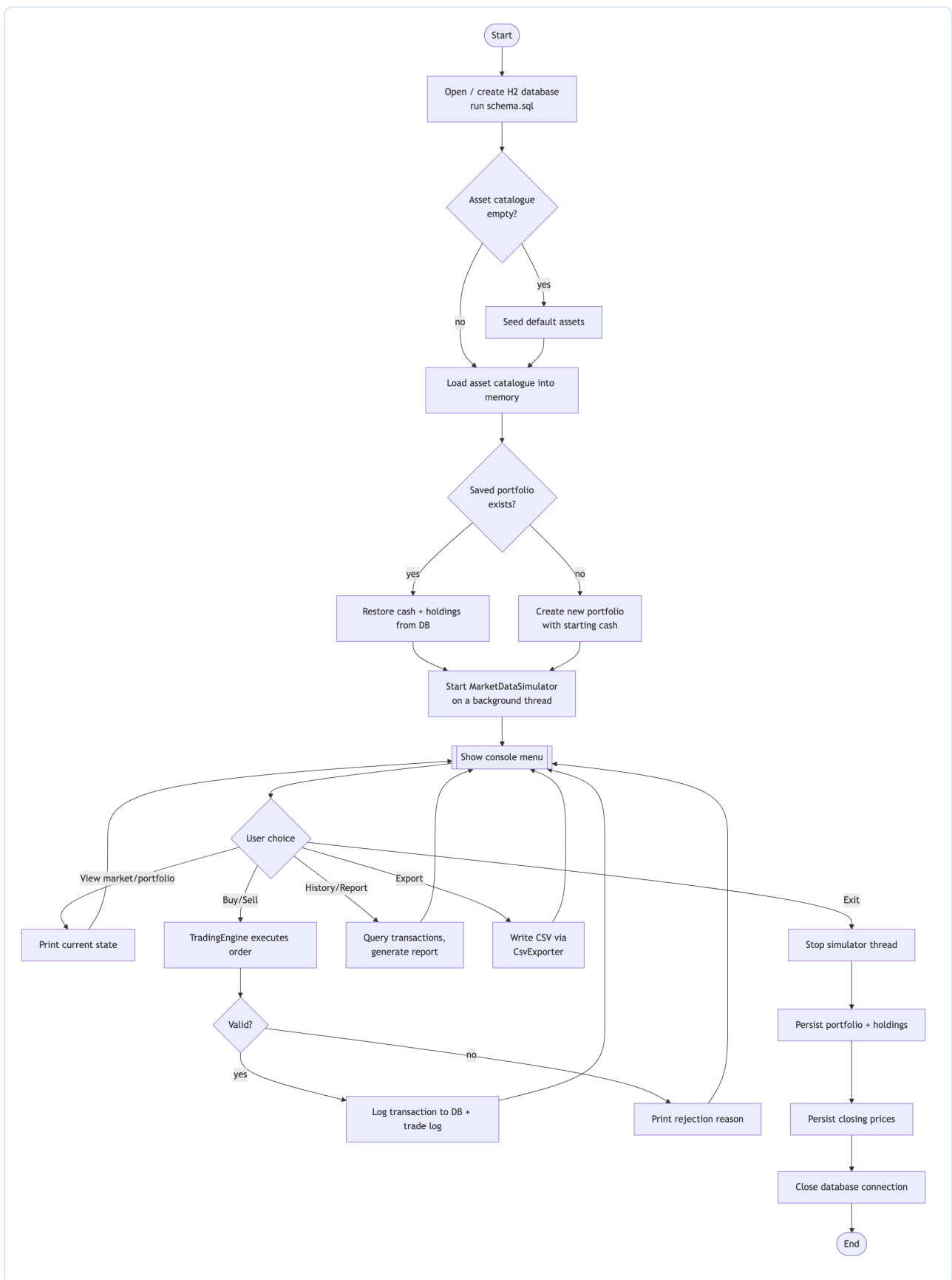


Figure 3 — Application lifecycle from launch to graceful shutdown.

6.3 Class Diagram

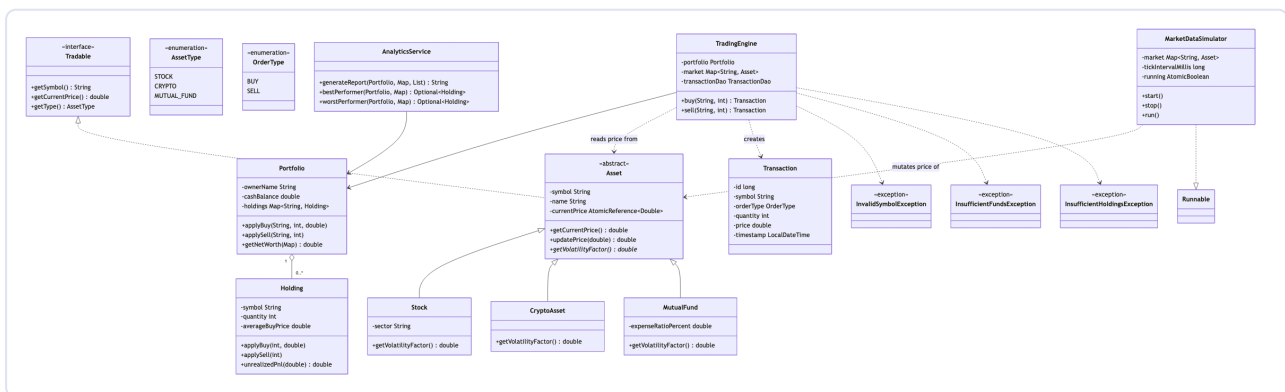


Figure 4 — Domain model and the services operating on it.

6.4 Sequence Diagram — Placing a Buy Order

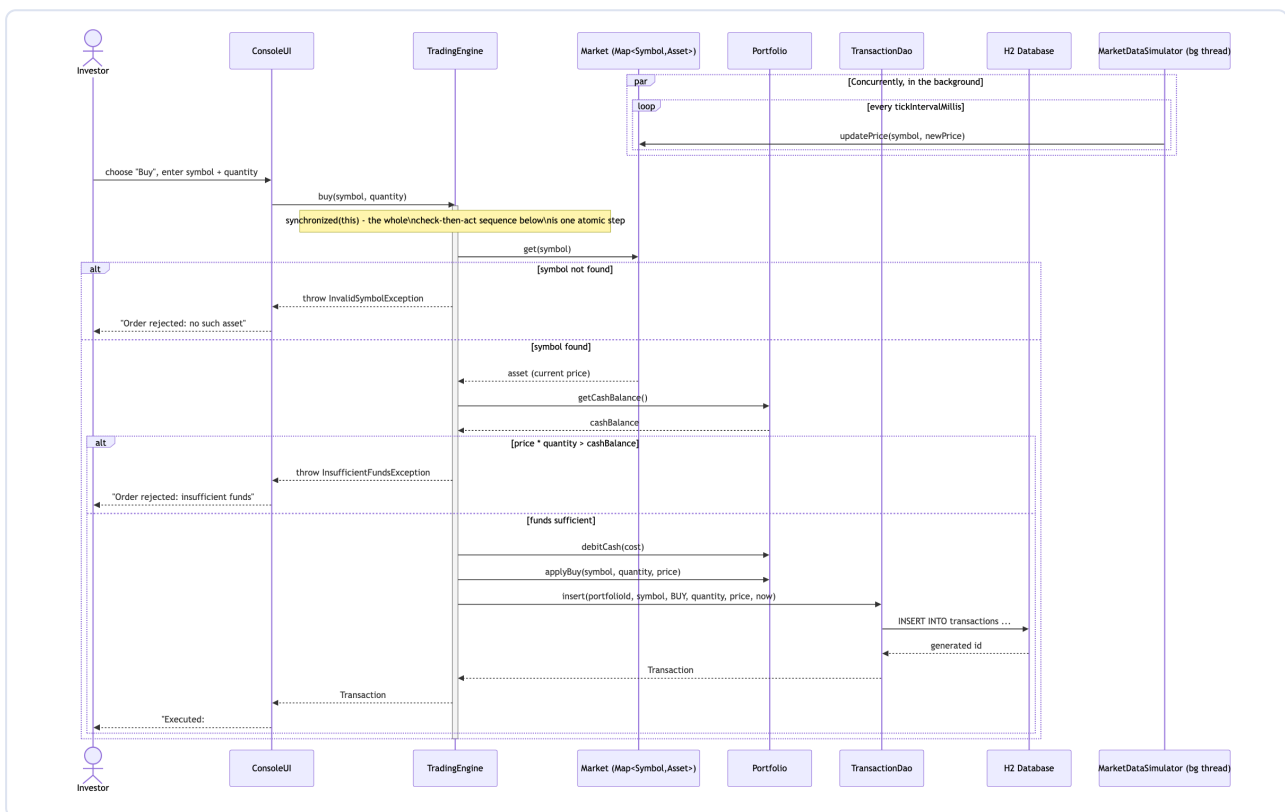


Figure 5 — The background simulator (top) runs concurrently with the order path; the engine's synchronized block is what keeps a trade's price-read-then-act sequence atomic despite that.

6.5 ER Diagram

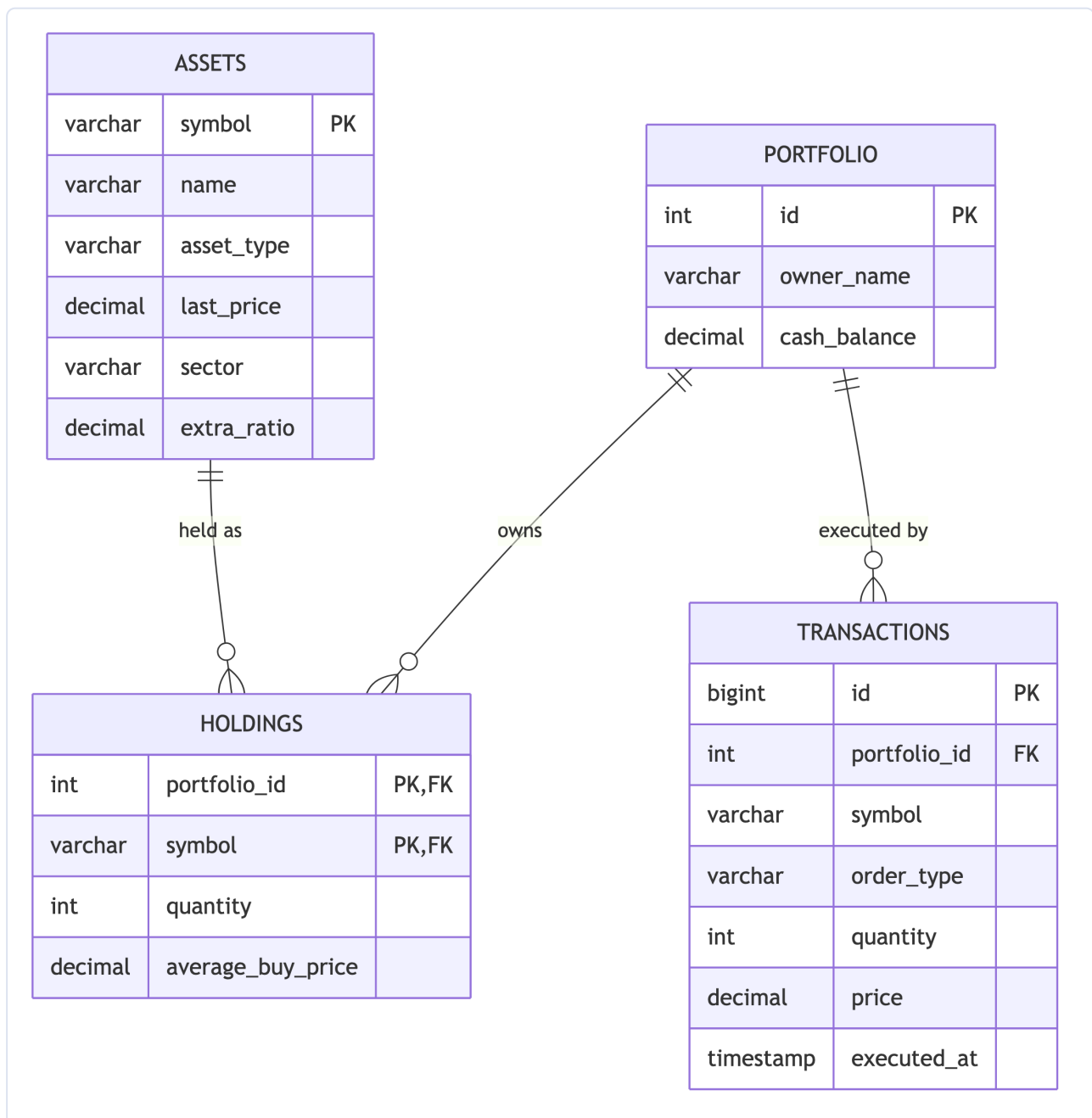


Figure 6 — Matches `src/main/resources/schema.sql` exactly.

7. Design Decisions & Rationale

7.1 Embedded H2 instead of MySQL/Postgres

The brief asks for JDBC-based persistence but does not mandate a specific engine. An embedded, file-based H2 database was chosen over a networked server so the project can be cloned and run with `mvn clean package && java -jar target/stockpulse.jar` alone — no separate database server to install, configure, or have running for a grader to reproduce the build.

7.2 Lock-free price publication via `AtomicReference`

An earlier draft stored each asset's price as a plain `double` field, synchronized on the asset for both reads and writes. That would have meant every single console read of a price (the menu prints market prices constantly) contends for the same lock the simulator thread acquires every tick. Since a price is a single immutable value being replaced, not accumulated, `AtomicReference<Double>` gives every reader a consistent, current value with no blocking and no torn reads — a deliberately narrower guarantee than a full lock, chosen because it's exactly the guarantee this problem needs.

7.3 Synchronizing the trading engine, not the market

A trade is a multi-step sequence — read price, check funds/holdings, mutate the portfolio, log the transaction — that must appear atomic even though the price itself may move again the instant after it's read. `TradingEngine.buy / sell` are therefore `synchronized` on the engine instance, so two orders can never interleave their check-then-act steps, while the simulator remains completely free to keep mutating prices without ever touching that lock.

7.4 Separating DAO save/load from the per-tick simulation loop

Persisting every price change would make the database a bottleneck for what is otherwise a lock-free update and would generate an enormous, useless write volume. Prices are instead loaded once at startup and snapshotted once at a graceful shutdown; only trades, which are comparatively rare and must never be lost, are written immediately as they occur. See §5, Figure 1.

7.5 Checked exceptions for rejected orders

`InvalidSymbolException`, `InsufficientFundsException`, and `InsufficientHoldingsException` are checked exceptions, not unchecked. A rejected trade is an expected, routine outcome of user input (unlike a real bug), and making it checked forces every call site — currently just `ConsoleUI`, but any future caller too — to explicitly decide how to present the rejection rather than letting it silently propagate.

8. Implementation Details

The codebase is organized into eight packages under `com.stockpulse` , totaling 19 source files plus 3 test classes (12 test methods):

Package	Classes	Purpose
<code>model</code>	<code>AssetType</code> , <code>OrderType</code> , <code>Tradable</code> , <code>Asset</code> , <code>Stock</code> , <code>CryptoAsset</code> , <code>MutualFund</code> , <code>Holding</code> , <code>Portfolio</code> , <code>Transaction</code>	The OOP domain: an abstract <code>Asset</code> implementing <code>Tradable</code> , specialized by three concrete asset types with different volatility bands, plus the portfolio/holding/transaction value objects.
<code>exception</code>	<code>InvalidSymbolException</code> , <code>InsufficientFundsException</code> , <code>InsufficientHoldingsException</code>	Custom checked exceptions for every way an order can be rejected.
<code>db</code>	<code>DatabaseManager</code>	Owns the single JDBC connection and bootstraps the schema from <code>schema.sql</code> on first run.
<code>dao</code>	<code>AssetDao</code> , <code>PortfolioDao</code> , <code>TransactionDao</code>	All SQL access, exclusively through <code>PreparedStatement</code> .
<code>market</code>	<code>MarketDataSimulator</code>	The background <code>Runnable</code> thread that drives price movement.
<code>service</code>	<code>TradingEngine</code> , <code>AnalyticsService</code>	Order execution and portfolio analytics — the two classes exercised most heavily by the test suite.
<code>io</code>	<code>CsvExporter</code> , <code>TradeLogger</code>	Character-stream file I/O (<code>BufferedWriter</code> / <code>FileWriter</code>), independent of the JDBC layer.
<code>cli</code>	<code>ConsoleUI</code>	The <code>Scanner</code> -driven menu loop and the only class that talks to <code>System.in / out</code> .

8.1 Key Concepts Demonstrated per Syllabus Unit

Syllabus Unit	Where in this project
Unit 1 — Flow control	Menu dispatch (<code>switch</code> expression in <code>ConsoleUI.run</code>), loops throughout DAOs and the simulator.
Unit 2 — OOP	Abstract classes, interfaces, inheritance, polymorphism, and enums across the <code>model</code> package.

Unit 3 — Exceptions & multithreading	Custom checked exceptions, multi-catch in <code>ConsoleUI</code> ; <code>MarketDataSimulator</code> as a dedicated daemon thread; <code>synchronized</code> methods in <code>TradingEngine</code> .
Unit 4 — Collections & I/O streams	<code>ConcurrentHashMap</code> for holdings/market, <code>List</code> for transaction history; <code>BufferedWriter</code> / <code>FileWriter</code> in the <code>io</code> package.
Unit 5 — JDBC	The entire <code>db</code> / <code>dao</code> packages.

9. Screenshots / Results

As a console application, "screenshots" are captured here as literal terminal transcripts from a real run (reproduced verbatim, not re-typed), which is a more precise record than a screen image for a text interface.

9.1 Startup and Market View

```
$ java -jar target/stockpulse.jar
=====
StockPulse - Multi-threaded Portfolio Simulator
Prices update live in the background every few
seconds while you trade.
=====

1) View market    2) View portfolio  3) Buy    4) Sell
5) Transaction history  6) Analytics report
7) Export holdings CSV  8) Export transactions CSV  0) Exit
> 1

BTC    Bitcoin                6182730.65  [CRYPTO]
ETH    Ethereum                300491.79  [CRYPTO]
HDFC    HDFC Bank Ltd           1663.36   [STOCK]
INFY    Infosys Ltd            1508.78   [STOCK]
NIFB    Nifty Bluechip Fund      250.48   [MUTUAL_FUND]
TATA    Tata Motors Ltd          961.58   [STOCK]
```

9.2 A Successful Buy, and an Order Rejected by Exception Handling

```
> 3
Symbol: INFY
Quantity: 20
Executed: #1 2026-09-05T23:48:30.348017 BUY INFY qty=20 price=1508.78
total=30175.60

> 3
Symbol: BTC
Quantity: 1
Order rejected: Order requires 6182730.65 but only 69824.40 is available
```

The second order is rejected by a genuine `InsufficientFundsException` thrown from `TradingEngine.buy` — not a hypothetical case, this is the actual exception message produced by the running program.

9.3 Sell, Transaction History, and the Analytics Report

```
> 4
Symbol: INFY
Quantity: 5
Executed: #2 2026-09-05T23:48:30.350098 SELL INFY qty=5 price=1508.78
total=7543.90
```

```
> 5
#1 2026-09-05T23:48:30.348017 BUY  INFY qty=20 price=1508.78 total=30175.60
#2 2026-09-05T23:48:30.350098 SELL INFY qty=5  price=1508.78 total=7543.90
```

```
> 6
```

```
===== Portfolio Report: Aadarsh =====
```

```
Cash balance:      77368.30
Holdings value:    22631.70
Net worth:         100000.00
Total trades:      2
```

SYM	QTY	AVG COST	PRICE	P&L
INFY	15	1508.78	1508.78	0.00

```
Best performer:  INFY (0.00%)
```

```
Worst performer: INFY (0.00%)
```


10. Testing Approach

The suite uses JUnit 5 and runs via `mvn test`. 12 tests across 3 classes, all passing:

Test class	What it verifies
<code>PortfolioTest</code> (4 tests)	Weighted-average cost-basis math across repeated buys; a holding is fully removed once its quantity reaches zero; net worth correctly combines cash and marked-to-market holdings; unrealized P&L sign and magnitude.
<code>TradingEngineTest</code> (6 tests)	A valid buy debits cash and creates a holding; a buy that costs more than available cash throws <code>InsufficientFundsException</code> ; buying/selling an unknown symbol throws <code>InvalidSymbolException</code> ; selling more than is held throws <code>InsufficientHoldingsException</code> ; a sell after a buy correctly reduces the holding and credits cash; every executed trade is actually persisted via <code>TransactionDao</code> .
<code>MarketDataSimulatorTest</code> (2 tests)	A single tick never moves any asset's price beyond its own volatility band; the background thread starts and stops cleanly (a real concurrency smoke test — this exercises the actual worker thread, not a mocked one).

`TradingEngineTest` runs against a fresh, isolated in-memory H2 database (`jdbc:h2:mem:<random-uuid>`) created in `@BeforeEach` and discarded in `@AfterEach`, so the test suite never touches, seeds, or corrupts the real `data/stockpulse.mv.db` file used by the actual application.

Manual end-to-end verification performed beyond the automated suite: a full scripted console session (buy, sell, export, exit, then relaunch) was run against the real packaged jar to confirm that the portfolio, holdings, and last-known prices are correctly restored from disk after a process restart — see §11 for a bug this manual pass actually caught.

11. Challenges Faced

11.1 A real shutdown-hook race condition

The first working version registered a single `Runtime.getRuntime().addShutdownHook(...)` to save the portfolio and close the database, and called `System.exit(0)` after the menu loop exited normally. Manual end-to-end testing (not the unit tests, which are too narrow to catch this) immediately produced:

```
Exception in thread "Thread-2" java.lang.IllegalStateException: Failed to save the
portfolio
Caused by: org.h2.jdbc.JdbcSQLNonTransientConnectionException: The database is open
in exclusive
mode; can not open additional connections
```

Root cause: the JVM runs all registered shutdown hooks *concurrently*, on separate threads, with no ordering guarantee — H2 registers its own internal shutdown hook, and that hook was racing against this project's save-and-close hook over the same connection. Calling `System.exit` after an already-complete, ordinary program exit doesn't avoid this: shutdown hooks fire on *any* JVM termination, not only an explicit `exit` call.

Fix: the save-and-close sequence is now invoked *synchronously, directly* on the main thread immediately after the menu loop returns — before the JVM begins any shutdown sequence at all. The registered shutdown hook still exists, but only as a fallback for an abrupt termination (Ctrl+C/kill), and an `AtomicBoolean` guard ensures the save-and-close logic never runs twice. See `Main.java` for the guarded `shutdownOnce` runnable.

11.2 Choosing a volatility model realistic enough to be meaningful

An early version applied the same percentage swing to every asset, which made the "best/worst performer" analytics feature meaningless — everything moved by roughly the same amount. Giving each asset type its own volatility band (mutual fund 0.5%, stock 2%, crypto 8% per tick) was a small change that made the simulation behave recognizably like the real domain it models, and gave the analytics module something genuine to report on.

12. Learnings & Key Takeaways

- A background thread mutating shared state needs a concurrency story for every field it touches, not just the ones that "feel" concurrent — the fix in §11.1 came from a JVM-level guarantee (shutdown hooks run concurrently) that has nothing to do with the application's own threads.
- `AtomicReference` is often a better fit than `synchronized` for a single, frequently-read, infrequently-replaced value — it was the difference between the simulator and the console contending for a lock on every price read versus never contending at all.
- Separating what must be synchronous and durable (a trade) from what can be eventually-consistent and in-memory (a price tick) is a persistence design decision, not just a performance one — get it wrong and either the database becomes a bottleneck or a real trade can be lost.
- Manual end-to-end testing caught a bug (§11.1) that the unit test suite structurally could not have: the race only exists at real JVM shutdown, a lifecycle event no in-process JUnit test exercises.

13. Future Enhancements

- Replace the console front-end with a JavaFX or Swing GUI, including a live-updating price ticker driven by the same `MarketDataSimulator`.
- Replace the simulated feed with a real market-data API (e.g. a free-tier stock/crypto price API) behind the same `Tradable` interface, so no other layer would need to change.
- Support multiple named portfolios per user, and eventually multiple concurrent users trading against one shared market — the current single-engine `synchronized` design was deliberately kept simple for one portfolio, and would need a per-portfolio lock strategy to extend safely.
- Add a limit-order type (execute only once the market crosses a target price) instead of only immediate market orders.

14. References

1. Herbert Schildt, *Java: The Complete Reference*, 11th Edition, Oracle Press, 2018. (Course textbook.)
2. Deitel & Deitel, *Java How to Program (Early Objects)*, 10th Edition, Pearson, 2015.
3. Cay S. Horstmann and Gary Cornell, *Core Java Vol. I – Fundamentals*, 8th Edition, Pearson, 2011.
4. Steven Holzner et al., *Java 2 Black Book*, Dreamtech Press, Reprint 2010.
5. Oracle, *Java Platform, Standard Edition Documentation* — `java.util.concurrent`, `java.sql`, and `java.io` package specifications.
6. H2 Database Engine documentation — h2database.com.
7. CSE2006 — Programming in Java, Course Handout, VIT Bhopal (Prepared by Dr. Lakshmi D; Compiled by Dr. M. Ashwin & Dr. Sandip Mal), recommended by the Board of Studies on 27.12.2021.